Notes on Coding Theory

1 Low-Density Parity-Check Codes

Low-density parity-check (LDPC) codes are a class of linear block codes with implementable decoders, which provide near-capacity performance on a large set of data-transmission and data-storage channels.

1.1 Representation of LDPC Codes

A low-density parity-check code is a linear block code given by the null space of an $m \times n$ parity-check matrix $\mathbf{H}$ that has a low density of 1s. The density need only be sufficiently low to permit effective iterative decoding.

$$\mathbf{H} \in \{0,1\}^{m \times n}, \qquad \mathbf{Ht} = \mathbf{0} \quad \text{for all codewords } \mathbf{t}$$

LDPC codes can be classified as:

- A regular LDPC code is a linear block code whose parity-check matrix $\mathbf{H}$ has fixed column weight g and row weight r, where $r/g = n/m$ and $g \ll m$. In this case, the code rate R is bounded by $R \geq 1 - \frac{m}{n} = 1 - \frac{g}{r}$ with equality when $\mathbf{H}$ is full rank.
- An irregular LDPC code is a linear block code whose parity-check matrix $\mathbf{H}$ has variable column weights and row weights, being determined by code-design procedures.

Additionally, almost all LDPC code constructions impose the structural property on $\mathbf{H}$: no two rows or two columns have more than one position in common that contains a nonzero element.

Graphically, **Tanner graphs** are used to represent a LDPC code from its parity-check matrix $\mathbf{H}$. Tanner graph is a bipartite graph, that is, a graph whose nodes may be separated into two types, with edges connecting only nodes of different types. The two types of nodes in a Tanner graph are the variable nodes (or code-bit nodes) and the check nodes (or constraint nodes), which are denoted by VNs and CNs, respectively.

In a Tanner graph, CN i is connected to VN j whenever element h_{ij} in $\mathbf{H}$ is a 1. Therefore, there are m CNs in a Tanner graph, one for each check equation, and n VNs, one for each code bit; the m rows of $\mathbf{H}$ specify the m CN connections, and the n columns of $\mathbf{H}$ specify the n VN connections. This is the basis of **iterative decoding**: each of the nodes acts as a locally operating processor and each edge acts as a bus that conveys information from a given node to each of its neighbors.

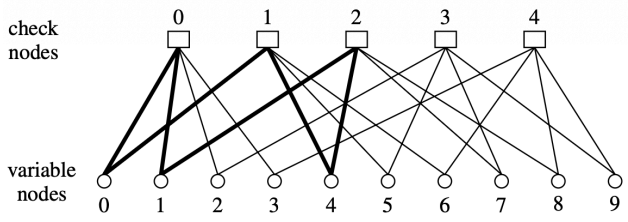

Figure 1: Tanner graph of a (10,5) linear block code with $r = 4$, $g = 2$ (i.e. the degree of each VN is 2 and the degree of each CN is 4)

The parity-check matrix for the Tanner graph above is

$$\mathbf{H} = \begin{bmatrix} 1 & 1 & 1 & 1 & 0 & 0 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 & 1 & 0 & 0 & 1 & 1 & 0 \\ 0 & 0 & 1 & 0 & 0 & 1 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 1 & 0 & 0 & 1 & 0 & 1 & 1 \end{bmatrix}.$$

As follows from $\mathbf{v}\mathbf{H}^T = \mathbf{0}$, that the bit values connected to the same check node must sum to zero (mod 2). Also, since $\text{rank}(\mathbf{H}) = 4 \neq 5$, $\mathbf{H}$ is not full rank, so the code rate is $R = 1 - 4/10 = 3/5$. As shown in 1, the six thickened edges form a **closed path**, which is called a **cycle**. The length of a cycle is equal to the number of edges which form the cycle: a cycle of length l is often called on l-cycle, and the minimum cycle length in a given bipartite graph is called the **graph's girth**.

It is possible to more closely approach capacity limits with irregular LDPC codes than with regular LDPC codes. For irregular LDPC codes, the parameters g and r vary with the columns and rows, so it is usual to specify the VN and CN **degree-distribution polynomials**. In the polynomial,

$$\lambda(X) = \sum_{d=1}^{d_v} \lambda_d X^{d-1},$$

λ_d denotes the fraction of all edges connected to degree-d VNs and d_v denotes the maximum VN degree. Similarly, in the polynomial

$$\rho(X) = \sum_{d=1}^{d_c} \rho_d X^{d-1},$$

ρ_d denotes the fraction of all edges connected to degree-d CNs and d_c denotes the maximum CN degree. Note that, for the regular code above, for which $g = 2$ and $r = 4$, we have $\lambda(X) = X$ and $\rho(X) = X^3$. It is also possible to view this from a node perpective, $L(X) = \sum_{d=1}^{d_v} L_d X^d$ and $R(X) = \sum_{d=1}^{d_c} R_d X^d$, where $L_d = \frac{\lambda_d/d}{\int_0^1 \lambda(X)\, dX}$ and $R_d = \frac{\rho_d/d}{\int_0^1 \rho(X)\, dX}$.

The average degrees are $\bar{d}_v = L'(1) = \left(\int_0^1 \lambda(x)\, dx\right)^{-1}$ and $\bar{d}_c = R'(1) = \left(\int_0^1 \rho(x)\, dx\right)^{-1}$. The design rate of an LDPC code is $R \geq 1 - \frac{\bar{d}_v}{\bar{d}_c} = 1 - \frac{L'(1)}{R'(1)} = 1 - \frac{\int_0^1 \rho(x)\, dx}{\int_0^1 \lambda(x)\, dx}$.

1.2 Message Passing and the Turbo Principle

Message-passing decoding refers to a collection of low-complexity decoders working in a distributed fashion to decode a received codeword in a concatenated coding scheme.

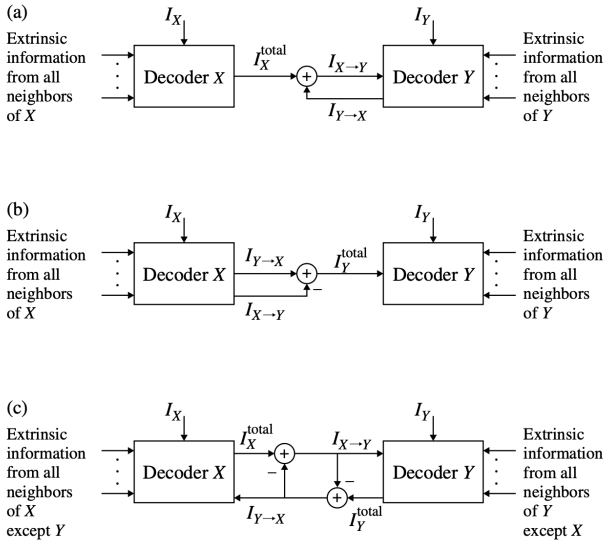

Figure 2: A depiction of the trubo principle for concatenated coding schemes.(a) Extrinsic information $I_{X\to Y}$ from decoder X to decoder Y. (b) Extrinsic information $I_{Y\to X}$ from decoder Y to decoder X. (c) A compact symmetric combination of (a) and (b).

The notion of **extrinsic-information passing** is called the turbo principle in the context of the iterative decoding of concatenated codes in communication channels. **Total information**

$$I_X^{\text{total}} = \sum_{Z\in N(X)} I_{Z\to X} + I_X$$

and the **extrinsic information**

$$I_{X\to Y} = \sum_{Z\in N(X)-\{Y\}} I_{Z\to X} + I_X$$

where I_X denotes the **intrinisic information** received from the channel. Essentially, Y receives the sum total of messages that X has available minus the information that Y already has, which means X does not pass to a neighboring decoder any information that the neighboring decoder already has.

1.3 Sum-Prudct Algorithm

The sum-product algorithm (SPA) or belief propagation algorithm (BPA) for general memoryless binary-input channels applies the turbo principle to correct the errors present in the received code bits.

In the context LDPC code, it can be considered as *a generalized concatenation of many Single-Parity-Check code (CNs) and Repetition code (VNs)*, and they work cooperatively in a distributed fashion to determine the correct code bit values.

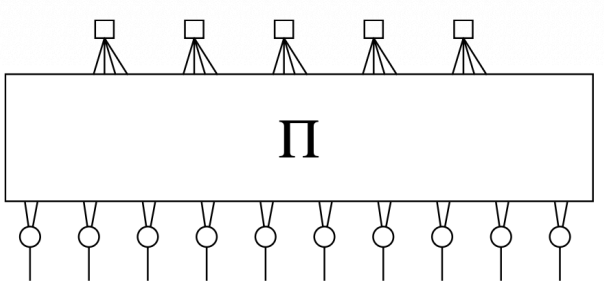

Figure 3: Graphical representation of an LDPC code as a concatenation of SPC and REP codes. π represents an interleaver.

The optimality criterion underlying the development of the SPA decoder is symbol-wise maximum a posteriori (MAP). we are interested in computing the a posteriori probability (APP) that a specific bit in the transmitted codeword $\mathbf{v} = [v_0\ v_1\ \cdots\ v_{n-1}]$ equals to 1, given the received codeword $\mathbf{y} = [y_0\ y_1\ \cdots\ y_{n-1}]$. For a general code bit v_j, the APP is

$$\Pr(v_j = 1 \mid \mathbf{y})$$

the APP ratio (or likelihood ratio under the uniform priors assumption),

$$l(v_j \mid \mathbf{y}) \triangleq \frac{\Pr(v_j = 0 \mid \mathbf{y})}{\Pr(v_j = 1 \mid \mathbf{y})}$$

or log-likelihood ratio (LLR) can be used for numerical stability,

$$L(v_j \mid \mathbf{y}) \triangleq \log\left(\frac{\Pr(v_j = 0 \mid \mathbf{y})}{\Pr(v_j = 1 \mid \mathbf{y})}\right)$$

The VN and CN decoders work cooperatively and iteratively to estimate $L(v_j \mid \mathbf{y})$ for $j = 0, 1, ..., n-1$. Throughout our development, the **flooding schedule** is employed. According to this schedule, all VNs process their inputs and pass extrinsic information up to their neighboring check nodes; the check nodes then process their inputs and pass extrinsic information down to their neighboring variable nodes; and the procedure repeats, starting with the variable nodes. After a preset maximum number of iterations of this VN/CN decoding round, or after some stopping criterion has been met, the decoder computes (estimates) the LLRs $L(v_j \mid \mathbf{y})$ from which decisions on the bits v_j are made. When the cycles are large, the estimates will be very accurate and the decoder will have near-optimal (MAP) performance.

SPA assumes the *LLR quantities received at each node from its neighbors are independent.* Clearly this breaks down when the *number of iterations exceeds half of the Tanner graph's girth.*

1.3.1 VN: Repetition Code MAP Decoder and APP Processor

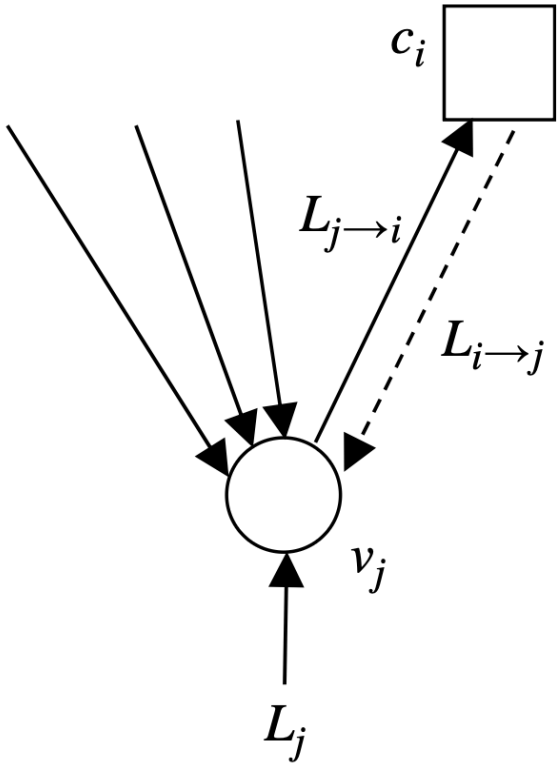

Figure 4: A VN decoder (REP decoder). VN j receives LLR information from the channel and from all of its neighbors, excluding message $L_{i\to j}$ from CN i, from which VN j composes message $L_{j\to i}$ that is sent to CN i

Consider a REP code in which the binary code symbol $c \in \{0, 1\}$ is transmitted over a memoryless channel d times so that the d-vector $\mathbf{r}$ is received. By definition, the MAP decoder computes the log-APP ratio

$$L(c \mid \mathbf{r}) = \log\left(\frac{\Pr(c = 0 \mid \mathbf{r})}{\Pr(c = 1 \mid \mathbf{r})}\right) = \log\left(\frac{\Pr(\mathbf{r} \mid c = 0)}{\Pr(\mathbf{r} \mid c = 1)}\right)$$

under an equally likely assumption for the value of c (ratio of a posterior = ratio of likelihood)

This simplifies as

$$\begin{aligned} L(c \mid \mathbf{r}) &= \log\left(\frac{\prod_{l=0}^{d-1} \Pr(r_l \mid c = 0)}{\prod_{l=0}^{d-1} \Pr(r_l \mid c = 1)}\right) \\ &= \sum_{l=0}^{d-1} \log\left(\frac{\Pr(r_l \mid c = 0)}{\Pr(r_l \mid c = 1)}\right) = \sum_{l=0}^{d-1} L(r_l \mid x) \end{aligned}$$

where $L(r_l \mid x)$ is obviously defined. Thus, the MAP receiver for a REP code computes the LLRs for each channel output r_l and adds them. *The MAP decision is* $\hat{c} = 0$ *if* $L(c \mid \mathbf{r}) \geq 0$ *and* $\hat{c} = 1$ *otherwise.*

In the context of LDPC decoding, the above expression is adapted to compute the extrinsic information to be sent from VN j to CN i of Figure 4,

$$\boxed{L_{j\to i} = L_j + \sum_{i'\in N(j)-\{i\}} L_{i'\to j}}$$

The quantity L_j in this expression is the LLR value computed from the channel sample y_j,

$$L_j = L(c_j \mid y_j) = \log\left(\frac{\Pr(c_j = 0 \mid y_j)}{\Pr(c_j = 1 \mid y_j)}\right).$$

At the last iteration, VN j produces a decision based on

$$L_j^{\text{total}} = L_j + \sum_{i\in N(j)} L_{i\to j}$$

1.3.2 CN: Single-Parity-Check Code MAP Decoder and APP Processor

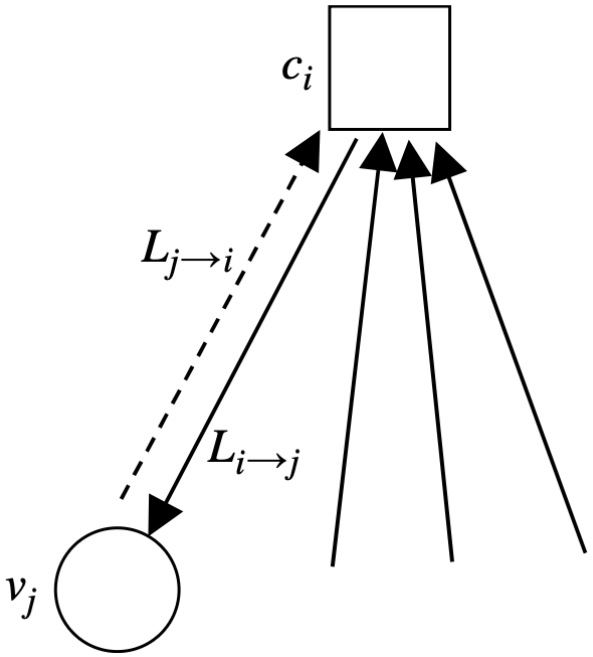

Figure 5: A CN decoder (SPC decoder). CN i receives LLR information from all of its neighbors, excluding message $L_{j\to i}$ from VN j, from which CN i composes message $L_{i\to j}$ that is sent to VN j

Lemma 1. *Consider a vector of d independent binary random variables $\mathbf{a} = [a_0, a_1, \ldots, a_{d-1}]$ in which $\Pr(a_l = 1) = p_1^{(l)}$ and $\Pr(a_l = 0) = p_0^{(l)}$. Then the probability that $\mathbf{a}$ contains an even number of 1s is*

$$\frac{1}{2} + \frac{1}{2}\prod_{l=0}^{d-1}\left(1 - 2p_1^{(l)}\right),$$

and the probability that **a** *contains an odd number of 1s is*

$$\frac{1}{2} - \frac{1}{2}\prod_{l=0}^{d-1}\left(1 - 2p_1^{(l)}\right).$$

Consider the transmission of a length-d SPC codeword $\mathbf{c}$ over a memoryless channel whose output is $\mathbf{r}$. The bits in the codeword $\mathbf{c}$ have a single constraint: *there must be an even number of 1s in* $\mathbf{c}$. Without loss of generality, we focus on bit c_0, for which the MAP decision rule is

$$\hat{c}_0 = \arg\max_{b\in\{0,1\}} \Pr(c_0 = b \mid \mathbf{r}, \text{SPC}),$$

where the conditioning on SPC is a reminder that there is an SPC constraint imposed on $\mathbf{c}$. Consider now

$$\begin{aligned}\Pr(c_0 = 0 \mid \mathbf{r}, \text{SPC}) &= \Pr(c_1, c_2, \ldots, c_{d-1} \text{ has an even number of 1s} \mid \mathbf{r}) \\ &= \frac{1}{2} + \frac{1}{2}\prod_{l=1}^{d-1}\left(1 - 2\Pr(c_l = 1 \mid r_l)\right),\end{aligned}$$

where the second line follows from the 1. Rearranging gives

$$1 - 2\Pr(c_0 = 1 \mid \mathbf{r}, \text{SPC}) = \prod_{l=1}^{d-1}\left(1 - 2\Pr(c_l = 1 \mid r_l)\right),$$

where $p(c_0 = 0 \mid \mathbf{r}, \text{SPC}) = 1 - p(c_0 = 1 \mid \mathbf{r}, \text{SPC})$.

We can change this to an **LLR representation** using the easily proven relation for a generic binary random variable with probabilities p_1 and p_0,

$$1 - 2p_1 = \tanh\left(\frac{1}{2}\log\left(\frac{p_0}{p_1}\right)\right) = \tanh\left(\frac{1}{2}\text{LLR}\right).$$

where here $\text{LLR} = \log(p_0/p_1)$. Applying this relation gives

$$\tanh\left(\frac{1}{2}L(c_0 \mid \mathbf{r}, \text{SPC})\right) = \prod_{l=1}^{d-1}\tanh\left(\frac{1}{2}L(c_l \mid r_l)\right)$$

or

$$L(c_0 \mid \mathbf{r}, \text{SPC}) = 2\tanh^{-1}\left(\prod_{l=1}^{d-1}\tanh\left(\frac{1}{2}L(c_l \mid r_l)\right)\right).$$

Thus, *the MAP decoder for bit* c_0 *in a length-d SPC code makes the decision* $\hat{c}_0 = 0$ *if* $L(c_0 \mid \mathbf{r}, \text{SPC}) \geq 0$ *and* $\hat{c}_0 = 1$ *otherwise.*

In the context of LDPC decoding, when the CNs function as APP processors instead of MAP decoders, CN i computes the extrinsic information

$$\boxed{L_{i\to j} = 2\tanh^{-1}\left(\prod_{j'\in N(i)-\{j\}}\tanh\left(\frac{1}{2}L_{j'\to i}\right)\right)}$$

and transmits it to VN j.

1.3.3 The Gallager SPA Decoder

Algorithm 1 The Gallager Sum-Product Algorithm

1: **Initialization:** For all j, initialize L_j according to the appropriate channel model. Then, for all i, j for which $h_{ij} = 1$, set $L_{j\to i} = L_j$.

2: **CN update:** Compute outgoing CN messages $L_{i\to j}$ for each CN using

$$L_{i\to j} = 2\tanh^{-1}\left(\prod_{j'\in N(i)-\{j\}} \tanh\left(\frac{1}{2}L_{j'\to i}\right)\right)$$
$$= \prod_{j'\in N(i)-\{j\}} \alpha_{j'i}\cdot\phi\left(\sum_{j'\in(i)-\{j\}}\phi(\beta_{j'i})\right)$$
$$\text{where}\quad \phi(x) = -\log[\tanh(x/2)] = \log(\frac{e^x+1}{e^x-1}),\quad \alpha_{ji} = \text{sign}(L_{j\to i}),\quad \beta_{ji} = \mid L_{j\to i}\mid$$

and then transmit them to the VNs. Note that $\phi(x) = \phi^{-1}(x)$ for $x > 0$, which is less challenging to compute compare with the product of the tanh and $\tanh^{-1}$ functions.

3: **VN update:** Compute outgoing VN messages $L_{j\to i}$ for each VN using

$$L_{j\to i} = L_j + \sum_{i'\in N(j)-\{i\}} L_{i'\to j},$$

and then transmit them to the CNs.

4: **LLR total:** For $j = 0, 1, \ldots, n-1$, compute

$$L_j^{\text{total}} = L_j + \sum_{i\in N(j)} L_{i\to j}.$$

5: **Stopping criteria:** For $j = 0, 1, \ldots, n-1$, set

$$\hat{v}_j = \begin{cases} 1, & \text{if } L_j^{\text{total}} < 0, \\ 0, & \text{otherwise,}\end{cases}$$

to obtain $\hat{\mathbf{v}}$. If $\hat{\mathbf{v}}\mathbf{H}^T = \mathbf{0}$ or the number of iterations equals the maximum limit, stop; otherwise, go to Step 2.

The decoder is initialized by setting all VN messages $L_{j\to i}$ equal to

$$L_j = L(v_j \mid y_j) = \log\left(\frac{\Pr(v_j = 0 \mid y_j)}{\Pr(v_j = 1 \mid y_j)}\right),$$

for all j, i for which $h_{ij} = 1$. Here, y_j represents the channel value that was actually received, that is, it is not a variable here. We consider the following special cases.

BEC. In this case, $y_j \in \{0, 1, e\}$ and we define $p = \Pr(y_j = e \mid v_j = b)$ to be the erasure

probability, where $b \in \{0,1\}$. Then it is easy to see that

$$\Pr(v_j = b \mid y_j) = \begin{cases} 1, & \text{when } y_j = b, \\ 0, & \text{when } y_j = b^c, \\ 1/2, & \text{when } y_j = e, \end{cases}$$

where b^c represents the complement of b. From this, it follows that

$$L(v_j \mid y_j) = \begin{cases} +\infty, & y_j = 0, \\ -\infty, & y_j = 1, \\ 0, & y_j = e. \end{cases}$$

BSC. In this case, $y_j \in \{0,1\}$ and we define $\varepsilon = \Pr(y_j = b^c \mid v_j = b)$ to be the error probability. Then it is obvious that

$$\Pr(v_j = b \mid y_j) = \begin{cases} 1-\varepsilon, & \text{when } y_j = b, \\ \varepsilon, & \text{when } y_j = b^c. \end{cases}$$

From this, we have

$$L(v_j \mid y_j) = (-1)^{y_j} \log\left(\frac{1-\varepsilon}{\varepsilon}\right).$$

Note that knowledge of ε is necessary.

BI-AWGNC. We first let $x_j = (-1)^{v_j}$ be the jth transmitted binary value; note that $x_j = +1(-1)$ when $v_j = 0(1)$. We shall use x_j and v_j interchangeably hereafter. The jth received sample is $y_j = x_j + n_j$, where the n_j are independent and normally distributed as $\mathcal{N}(0, \sigma^2)$. Then it is easy to show that

$$\Pr(x_j = x \mid y_j) = \left[1 + \exp\left(-2y_j x/\sigma^2\right)\right]^{-1},$$

where $x \in \{\pm 1\}$ and, from this, that

$$L(v_j \mid y_j) = 2y_j/\sigma^2.$$

In practice, an estimate of σ^2 is necessary.

Rayleigh. Assuming an independent Rayleigh fading channel and the decoder has perfect channel state information. The model is similar to that of the AWGNC: $y_j = \alpha_j x_j + n_j$, where $\{\alpha_j\}$ are independent Rayleigh random variables with unity variance. The channel transition probability in this case is

$$\Pr(x_j = x \mid y_j) = \left[1 + \exp\left(-2\alpha_j y_j x/\sigma^2\right)\right]^{-1}.$$

Then the variable nodes are initialized by

$$L(v_j \mid y_j) = 2\alpha_j y_j/\sigma^2.$$

In practice, estimates of α_j and σ^2 are necessary.

1.3.4 The Min-Sum Decoder

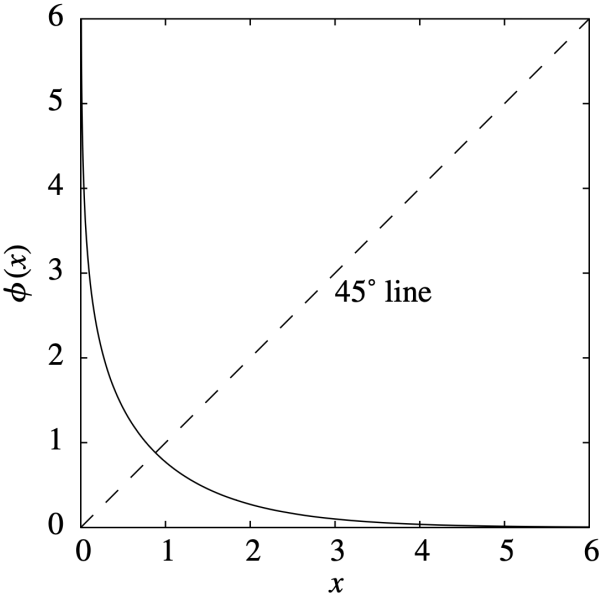

Figure 6: A plot of $\phi(x)$

Consider the CN update equation in the Gallager SPA, from the shape of $\phi(x)$ that the largest term in the sum corresponds to the smallest β_{ji} so that, assuming that this term dominates the sum,

$$\phi\left(\sum_{j'}\phi(\beta_{j'i})\right) \simeq \phi\left(\phi\left(\min_{j'}\beta_{j'i}\right)\right) = \min_{j'\in N(i)-\{j\}}\beta_{j'i}$$

Thus, the min-sum algorithm is simplt the log-domain SPA with Step 2 replaces by

$$\text{CN update: } L_{i\to j} = \prod_{j'\in N(i)-\{j\}}\alpha_{j'i}\cdot\min_{j'\in N(i)-\{j\}}\beta_{j'i}.$$

Note the fact that *the sign of LLR represents the estimated code bit value (0 or 1) and the magnitude of LLR represents the confidence or reliability level of that estimate.* Therefore, the min-sum algorithm says that the CNs are only as confident as the least confident input.

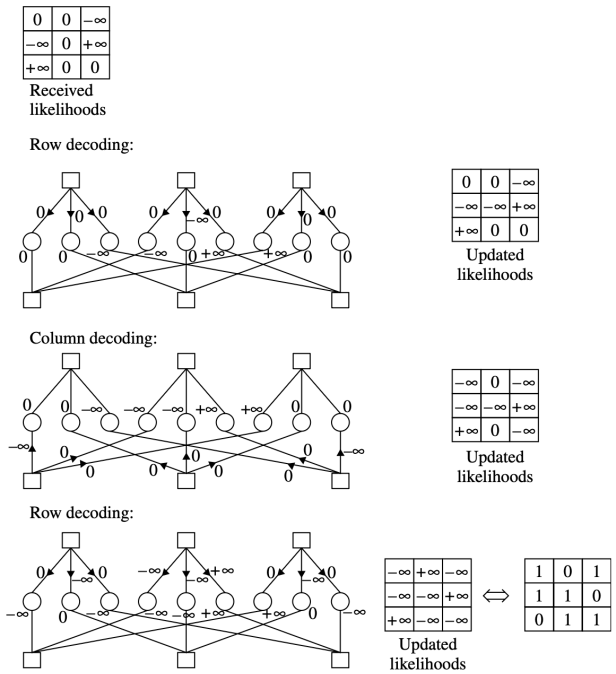

Figure 7: An example of min-sum decoding on the BEC using the (3,2) regular LDPC code. Note that,for BEC, $\phi\left(\sum_{j'} \phi(\beta_{j'i})\right)$ is 0 or ∞ exactly when $min_{j'}\beta_{j'i}$ is 0 or ∞, respectively. Note also that this is equivalent to the iterative erasure filling algorithm.

Experimentally, for the first iteration of a min-sum decoder, the extrinsic information passed from a CN to a VN for a min-sum decoder is on average larger in magnitude than the extrinsic information that would have been passed by an SPA decoder. That is, the min-sum decoder is generally too optimistic in assigning reliabilities. Thus, we can use

$$\text{CN update: } L_{i\to j} = \prod_{j'\in N(i)-\{j\}} \alpha_{j'i} \cdot c_{atten} \cdot \min_{j'\in N(i)-\{j\}} \beta_{j'i}.$$

where $0 < c_{atten} < 1$ is the arrenuation factor, usually $c_{atten} = 0.5$.

An alternative technique for mitigating the overly optimistic extrinsic information that occurs in a min-sum decoder is to subtract a constant offset $c_{offset} > 0$ from each CN-to-VN message magnitude $|L_{i\to j}|$, subject to the constraint that the result is non-negative. Thus, the offset-min-sum algorithm computes

$$\text{CN update: } L_{i\to j} = \prod_{j'\in N(i)-\{j\}} \alpha_{j'i} \cdot \max\left\{\min_{j'\in N(i)-\{j\}} \beta_{j'i} - c_{offset},\ 0\right\}.$$

1.4 Decoding Algoithms for the BEC and the BSC

The BEC and BSC are studied as special cases of the general SPA because their discrete channel outputs simplify the message-passing framework to simple symbolic operations. In the general SPA for the AWGN channel, received samples $y_j \in \mathbb{R}$ produce continuously-valued LLRs, and all Tanner graph messages are real-valued, requiring full nonlinear tanh-based computations at each node.

On the **BEC**, the LLR takes only three values $\{-\infty, 0, +\infty\}$, and the SPA reduces to iterative erasure filling — a check node resolves an erasure only if exactly one neighbor is erased, collapsing

the check node update to a simple XOR. Decoding failure is governed entirely by **stopping sets**, which cause failure even under ML decoding and are therefore a fundamental limitation of the code itself.

On the **BSC**, every received bit carries the same fixed-magnitude LLR $\log \frac{1-\varepsilon}{\varepsilon}$, providing no graded soft information. Gallager's **Algorithm A and B** exploit this by discarding soft information entirely, passing only binary hard decisions with XOR-based check node updates, achieving very low complexity at the cost of performance relative to full SPA.

1.4.1 Iterative Erasure Filling for the BEC

The iterative decoding algorithm for the BEC simply works to fill in the erased bits in such a way that all of the check equations contained in $\mathbf{H}$ are eventually satisfied.

Algorithm 2 Iterative Erasure Decoding Algorithm

1: Find all check equations (rows of $\mathbf{H}$) involving a single erasure and solve for the values of bits corresponding to these erasures.
2: Repeat Step 1 until there exist no more erasures or until all equations are unsolvable.

If no parity-check equation that involves only one of the erasures in the erasure set can be found, then none of the equations will be solvable and decoding will stop. Code bit sets that lead to unsolvable equations when they are erased are called **stopping sets**. Thus, a subset S of code bits forms a stopping set if each equation that involves the bits in S involves two or more of them. In Tanner graph, S is a stopping set if all of its neighbors are connected to S at least twice. Such a configuration is problematic for an iterative decoder because, if these bits are erased, all equations involved will contain at least two erasures.

1.4.2 ML Decoder for the BEC

Adapted-$\mathbf{H}$ algorithm provides an example in which the stopping set in the previous example is resolvable.

Suppose codeword $\mathbf{c}$ is transmitted over a BEC and the received word $\mathbf{r}$ has elements in $\{0, 1, e\}$. Let J denote the index set of unerased positions and J' the index set of erased positions. On the BEC, the ML criterion simplifies dramatically. The ML decoder selects the codeword $\hat{\mathbf{c}} \in \mathcal{C}$ maximising $P(\mathbf{r} \mid \mathbf{c})$, but since any codeword disagreeing with a received unerased bit has zero probability, ML reduces to finding the *unique codeword consistent with all unerased positions*:

$$c_j = r_j \quad \forall\, j \in J$$

Combined with the codeword validity constraint $\mathbf{c}\mathbf{H}^T = \mathbf{c}_{J'}\mathbf{H}_{J'}^T + \mathbf{c}_J\mathbf{H}_J^T = \mathbf{0}$, the ML criterion is fully expressed by the linear system:

$$\mathbf{c}_{J'}\mathbf{H}_{J'}^T = \mathbf{c}_J\mathbf{H}_J^T$$

The right-hand side is fully known from the received word, so this is a linear system over GF(2) in the unknown $\mathbf{c}_{J'}$. A unique solution exists if and only if the rows of $\mathbf{H}_{J'}^T$ are linearly independent, which requires $|J'| \leq n - k$. Since any $d_{\min} - 1$ rows of $\mathbf{H}^T$ are linearly independent, uniqueness is guaranteed whenever $|J'| < d_{\min}$.

To solve the linear system $\mathbf{c}_{J'}\mathbf{H}_{J'}^T = \mathbf{c}_J\mathbf{H}_J^T$ for the unknown erased bits $\mathbf{c}_{J'}$, apply Gaussian elimination to $\mathbf{H}$ targeting the erased columns J', producing a modified matrix:

$$\tilde{\mathbf{H}} = \begin{bmatrix} \tilde{\mathbf{H}}_J & \tilde{\mathbf{H}}_{J'} \end{bmatrix}$$

where $\tilde{\mathbf{H}}_J$ is the submatrix of $\tilde{\mathbf{H}}$ corresponding to the unerased columns J modified by the elimination, and $\tilde{\mathbf{H}}_{J'}$ is the submatrix of $\tilde{\mathbf{H}}$ corresponding to the erased columns J', which after elimination takes the form:

$$\tilde{\mathbf{H}}_{J'} = \begin{bmatrix} \mathbf{T} \\ \mathbf{M} \end{bmatrix}$$

where $\mathbf{T}$ is a $|J'| \times |J'|$ lower-triangular matrix with ones on the diagonal, and $\mathbf{M}$ is an arbitrary binary matrix (which may be empty). The unknown erased bits $\mathbf{c}_{J'}$ are then recovered one at a time by back-substitution through the top $|J'|$ parity-check equations corresponding to $\mathbf{T}$, with the right-hand side fully computable from the known received bits $\mathbf{c}_J = \mathbf{r}_J$.

The system then becomes:

$$\mathbf{c}_{J'}\mathbf{T}^T = \mathbf{c}_J\tilde{\mathbf{H}}_J^T$$

Since $\mathbf{T}$ is lower-triangular with ones on the diagonal, the unknown erased bits $\mathbf{c}_{J'}$ are recovered one at a time by back-substitution through the parity-check equations of $\tilde{\mathbf{H}}$.

Thus, ML decoding on the BEC reduces entirely to Gaussian elimination over GF(2) — a linear algebraic operation with no probabilistic search required.

1.4.3 Gallager's Algorithm A and B for the BSC

As LLRs are trivial in BSC model, we use $M_{j\to i}$ and $M_{i\to j}$ to denote the LLR messages being passed $L_{j\to i}$ and $L_{j\to i}$.

Check node update equation for both algorithm A and B is

$$M_{i\to j} = \bigoplus_{j'\in N(i)-\{j\}} M_{j'\to i}$$

For algorithm A, the variable node update equation is

$$M_{j\to i} = \begin{cases} M_j, & \text{if } \exists\, i' \in N(j) - \{i\} \text{ s.t. } M_{i'\to j} = M_j, \\ M_j^c, & \text{otherwise.} \end{cases}$$

where the outgoing message $M_{j\to i}$ is set to M_j unless all of the incoming extrinsic messages $M_{i'\to j}$ disagree.

For algorithm A, the variable node update equation is

$$M_{j\to i} = \begin{cases} M_j^c, & \text{if } \left|\left\{i' \in N(j) - \{i\} : M_{i'\to j} = M_j^c\right\}\right| \geq t, \\ M_j, & \text{otherwise.} \end{cases}$$

which means as long as there are more than t incoming extrinsic information disagree with M_j then the outgoing $M_{j\to i}$ is set to M_j^c

The code bit decisions for these algorithms are majority-based, as follows. If the degree of a variable node is odd, then the decision is set to the majority of the incoming check node messages. If the degree of a variable node is even, then the decision is set to majority of the check node messages and the channel message.

1.4.4 The Bit-Flipping Algorithm for the BSC

Noting that *failed parity-check equations* are indicated by the elements of the syndrome, $\mathbf{s} = \mathbf{rH^T}$, and that the *number of failed parity checks for each code bit* is contained in the n-vector $\mathbf{f} = \mathbf{sH}$ (operations over the integers). If the size of $\mathbf{H}$ is $m \times n$, then the size of $\mathbf{r}$, $\mathbf{s}$, $\mathbf{f}$ is $1 \times n$, $1 \times m$, $1 \times n$ respectively.

Algorithm 3 The Bit-Flipping Algorithm

1: Compute $\mathbf{s} = \mathbf{rH}^T$ (operations over $\mathbb{F}^2$). If $\mathbf{s} = \mathbf{0}$, stop, since $\mathbf{r}$ is a codeword.
2: Compute $\mathbf{f} = \mathbf{sH}$ (operations over $\mathbb{Z}$).
3: Identify the elements of $\mathbf{f}$ greater than some preset threshold, and then flip all the bits in $\mathbf{r}$ corresponding to those elements.
4: If the maximum number of iterations has not been reached, go to Step 1 using the updated $\mathbf{r}$.

1.5 Generalized LDPC Codes

In a standard LDPC code, the two types of nodes in the Tanner graph perform simple operations. **Variable nodes (VN)** act as **repetition (REP) codes**, repeating the same bit value across all connected edges — a degree-d_v variable node is equivalent to a rate-$1/d_v$ repetition code. **Check nodes (CN)** act as **single parity-check (SPC) codes**, enforcing that the XOR of all connected bits equals zero. A standard LDPC code can therefore be viewed as a graph-based concatenation of REP and SPC component codes.

A **Generalized LDPC (GLDPC)** code replaces these simple component codes with stronger linear block codes. Check nodes are replaced by arbitrary linear block codes (e.g. Hamming, BCH, or Reed–Solomon codes) that enforce richer constraints than a single parity check, while variable nodes may similarly be replaced by stronger codes beyond simple repetition. The belief propagation (BP) decoder naturally extends to GLDPC codes: the simple REP and SPC message-passing rules at each node are replaced by the **a posteriori probability (APP) processor** for the corresponding component code, while the overall graph structure and message-passing framework remain unchanged.

The primary advantage of GLDPC codes is improved **minimum distance** for finite block lengths, which directly reduces the error floor, along with faster convergence due to stronger per-iteration information exchange. The cost is increased decoding complexity per node, since each check node now runs a full sub-decoder rather than a simple XOR operation.